Complete DSA Cheatsheet - 26 Problems

Trees, Graphs, Backtracking, DP, Strings, Lists

Core Concepts

References vs Values

Objects = References: Arrays, lists, nodes, TreeNode are shared across method calls
Primitives = Values: int, char, boolean are copied
Impact:

- Modifying array/list affects original
- Must explicitly copy: `new ArrayList<>(list)`
- Backtracking: MUST undo changes
- Strings: Immutable, safe to pass around

```
int[] arr = new int[5];
helper(arr);  // arr is modified!

List<Integer> list = new ArrayList<>();
backtrack(list);  // Must undo changes

String s = "abc";
helper(s + "d");  // Creates new string, s
    unchanged
```

DFS vs BFS

DFS (Stack/Recursion):

- Space: O(h) - height/depth
- Use: All paths, backtracking, topological sort
- Goes deep before wide
- Can get stuck in infinite depth

BFS (Queue):

- Space: O(w) - max width/breadth
- Use: Shortest path, level-order, min depth
- Goes wide before deep
- Guaranteed shortest in unweighted graph

When to use which:

- Need shortest? BFS
- Need all solutions? DFS
- Tree height > width? BFS saves space
- Tree width > height? DFS saves space

DP Fundamentals

Top-Down (Memoization):

- Start: Big problem → break down
- Method: Recursion + cache (HashMap/array)
- Pros: Natural, only computes needed states
- Cons: Recursion overhead, stack space O(n)

Bottom-Up (Tabulation):

- Start: Base cases → build up
- Method: Iteration + dp[] array
- Pros: No recursion, predictable, faster
- Cons: May compute unneeded states

Space Optimization:

- When: dp[i] only depends on fixed previous states
- Example: dp[i] = f(dp[i-1], dp[i-2])
- Use: 2-3 variables instead of array
- Cannot optimize: Unbounded knapsack, LCS

Pattern Recognition:

- Overlapping subproblems? DP candidate
- Optimal substructure? DP works
- Making choices? Consider DP

Backtracking Pattern

Core Template:

```
void backtrack(state, choices) {
    if (complete) {
        save_solution(state);  // COPY!
        return;
    }
    for (choice : choices) {
        if (valid(choice)) {
            make_choice(state);
            backtrack(new_state);
            undo_choice(state);  // RESTORE!
        }
    }
}
```

Key Points:

- MUST undo state changes
- MUST copy when saving solution
- Pruning: Skip invalid choices early
- String concat: No undo needed (immutable)

Common Patterns:

- Permutations: Track used elements
- Combinations: Start index to avoid duplicates
- Subsets: Save at every node, not just leaves

Tree Traversal Summary

Preorder: Root → Left → Right

- Use: Copy tree, serialize, expression tree prefix
- Stack iterative: Push right then left

Inorder: Left → Root → Right

- Use: BST gives sorted order
- Iterative: Tricky, go left first, then right

Postorder: Left → Right → Root

- Use: Delete tree, calculate tree properties
- Iterative: Reverse of modified preorder

Level Order (BFS): Level by level

- Use: Level problems, min depth, right view
- Must capture size before loop!

Essential Data Structures

```java
// Map iteration - Entry is efficient
for (Map.Entry<K,V> e : map.entrySet()) {
    K key = e.getKey();
    V val = e.getValue();
}

// PriorityQueue - min heap by default
PriorityQueue<Integer> pq = new PriorityQueue<>();
// Max heap: new PriorityQueue<>(Collections.
    reverseOrder())
pq.offer(5);
int min = pq.poll();  // Remove and return
int peek = pq.peek(); // Just look

// List of Lists - MUST copy!
List<List<Integer>> result = new ArrayList<>();
List<Integer> current = new ArrayList<>();
result.add(new ArrayList<>(current)); // Copy!
// NOT: result.add(current); // Wrong!

// HashMap with List values
Map<Integer, List<Integer>> map = new HashMap<>();
map.computeIfAbsent(key, k -> new ArrayList<>()).
    add(val);
// Replaces: if (!map.containsKey(key)) map.put(
    key, new ArrayList<>());

// Stack operations
Stack<Integer> stack = new Stack<>();
stack.push(5);
int top = stack.pop();
int peek = stack.peek();
boolean empty = stack.isEmpty();

// Queue operations (LinkedList implements Queue)
Queue<Integer> q = new LinkedList<>();
q.offer(5);  // add
int front = q.poll();  // remove
int peek = q.peek();
```

Critical Mistakes to Avoid

× `for (i < q.size())` in BFS - size changes!
✓ `int size = q.size(); for (i < size)`
× `result.add(current)` - adds reference!
✓ `result.add(new ArrayList<>(current))`
× Forget to undo in backtracking
✓ Add → Recurse → Remove
× `int min, int max` for BST validation
✓ `long min, long max` (node can be MIN/MAX)
× Return -1 in middle of DP recursion
✓ Use MAX_VALUE, convert to -1 at end
× `dp[i]` uses `array[i]` (off by one!)
✓ `dp[i]` uses `array[i-1]` or adjust indexing
× Mark visited AFTER recursive call
✓ Mark BEFORE to prevent infinite loop
× Use Integer.MAX_VALUE then add 1 (overflow!)
✓ Use n+1 or amount+1 as "infinity"
× Forward iteration in 0/1 Knapsack
✓ BACKWARD to prevent item reuse

1. Letter Combinations (17)

Phone digits → all letter combinations

Time: $O(4^n)$ **Space:** $O(n)$

Each digit has ≤4 letters. Total combos = 4^n. Recursion depth = n for call stack.

```java
String[] map = {"", "", "abc", "def", "ghi",
    "jkl", "mno", "pqrs", "tuv", "wxyz"};

public List<String> letterCombinations(String digits
    ) {
    List<String> res = new ArrayList<>();
    if (digits.isEmpty()) return res;  // Edge case!
    backtrack(0, "", digits, res);
    return res;
}

void backtrack(int i, String cur, String digits,
    List<String> res) {
    if (cur.length() == digits.length()) {
        res.add(cur);  // String is immutable, safe
    to add
        return;
    }
    String chars = map[digits.charAt(i) - '0'];
    for (char c : chars.toCharArray()) {
        backtrack(i + 1, cur + c, digits, res);
        // No undo needed - strings are immutable!
    }
}
```

! Strings are immutable - no undo needed!
! Empty string input returns empty list, not [""]

! Index i in recursion, NOT cur.length()

2. Permutations (46)

All arrangements of distinct integers

Time: O(n!) **Space:** O(n)

n! permutations, each takes O(n) to build/copy. Recursion stack depth = n.

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new ArrayList<>(),
        new boolean[nums.length], result);
    return result;
}

void backtrack(int[] nums, List<Integer> cur,
    boolean[] used, List<List<Integer>> result) {
    if (cur.size() == nums.length) {
        result.add(new ArrayList<>(cur));  // MUST COPY!
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;  // Skip used

        cur.add(nums[i]);       // Choose
        used[i] = true;

        backtrack(nums, cur, used, result);  // Explore

        cur.remove(cur.size() - 1);  // Unchoose
        used[i] = false;             // MUST UNDO BOTH!
    }
}
```

! MUST copy list: `new ArrayList<>(cur)`
! MUST undo BOTH add and used[i]!
! For duplicates (46 II), sort + skip consecutive same

3. Subsets (78)

All possible subsets (power set)

Time: O(n × 2^n) **Space:** O(n)

2^n subsets total. Each subset takes O(n) to copy. Recursion depth = n.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

void backtrack(int[] nums, int start,
    List<Integer> cur, List<List<Integer>> result) {
    // Save at EVERY node, not just leaves!
    result.add(new ArrayList<>(cur));

    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);
        backtrack(nums, i + 1, cur, result);  // i+1, NOT start+1
        cur.remove(cur.size() - 1);
    }
}
```

! Save at EVERY level, not just complete subsets!
! Use i+1, NOT start+1 in recursion!
! Empty subset [] is automatically saved first

4. Level Order Traversal (102)

Binary tree nodes grouped by level

Time: O(n) **Space:** O(w)

Visit each node once. Queue holds at most width (worst case: n/2 at last level).

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();  // MUST CAPTURE BEFORE LOOP!
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);

            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }

        result.add(level);  // New list each level
    }
    return result;
}
```

! Capture size BEFORE loop! Queue size changes inside loop!
! Create NEW list for each level, don't reuse!
! Check null before offering to queue!

5. Diameter of Binary Tree (543)

Longest path between any two nodes (counting edges)

Time: O(n) **Space:** O(h)

Visit each node once, O(1) work per node. Recursion stack = height h.

```
class Solution {
    int diameter = 0;  // Global to track max

    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }

    int height(TreeNode node) {
        if (node == null) return 0;

        int left = height(node.left);
        int right = height(node.right);

        // Update diameter at this node
        diameter = Math.max(diameter, left + right);

        // Return height to parent
        return 1 + Math.max(left, right);
    }
}
```

! Function returns HEIGHT, tracks DIAMETER separately!
! Diameter may NOT pass through root - check every node!
! Counting edges, not nodes! If nodes: return left + right + 1

6. Validate Binary Search Tree (98)

Check if tree satisfies BST property

Time: O(n) **Space:** O(h)

Visit each node once for O(1) check. Recursion stack depth = height h.

```
public boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

boolean valid(TreeNode node, long min, long max) {
    if (node == null) return true;

    // Current node must be strictly between min and max
    if (!(min < node.val && node.val < max)) {
        return false;
    }

    // Left subtree: all values < node.val
    // Right subtree: all values > node.val
    return valid(node.left, min, node.val) &&
           valid(node.right, node.val, max);
}
```

! Use LONG not int! Node values can be Integer.MIN/MAX!

! Strict inequality! No duplicates in BST!

! Pass node.val as bound, not node itself!

Alternative - Inorder gives sorted:

```
Integer prev = null;  // Use Integer for null check
public boolean isValidBST(TreeNode root) {
    if (root == null) return true;
    if (!isValidBST(root.left)) return false;
    if (prev != null && root.val <= prev) return false;
    prev = root.val;
    return isValidBST(root.right);
}
```

7. Lowest Common Ancestor (236)

Find LCA of two nodes in binary tree

Time: O(n) **Space:** O(h)

Worst case visits all nodes. Recursion stack = height h.

```
public TreeNode lowestCommonAncestor(TreeNode root,
    TreeNode p, TreeNode q) {
    // Base case: reached leaf or found target
    if (root == null) return null;
    if (root == p || root == q) return root;

    // Search both subtrees
    TreeNode left = lowestCommonAncestor(root.left,
    p, q);
    TreeNode right = lowestCommonAncestor(root.right
    , p, q);

    // Both found in different subtrees - current
    node is LCA
    if (left != null && right != null) return root;

    // Return whichever is not null
    return left != null ? left : right;
}
```

3 Cases: (1) Both in left, (2) Both in right, (3) Split across

! Early return when root matches p or q!

! Assumes p and q EXIST in tree!

! Works because we search bottom-up!

8. Binary Tree Right Side View (199)

View tree from right side (rightmost node at each level)

Time: O(n) **Space:** O(w)

Visit all n nodes. Queue width = w (max nodes at one level).

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        TreeNode rightSide = null;

        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node != null) {
                rightSide = node;  // Keep updating
                q.offer(node.left);
                q.offer(node.right);
            }
        }

        if (rightSide != null) res.add(rightSide.val
    );
    }
    return res;
}
```

Keep updating - last in level = rightmost!

! Can add nulls to queue OR check before adding

Alternative - DFS (right first):

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    dfs(root, 0, res);
    return res;
}

void dfs(TreeNode node, int depth, List<Integer> res
    ) {
    if (node == null) return;
    if (depth == res.size()) {  // First time at
    this depth
        res.add(node.val);
    }
    dfs(node.right, depth + 1, res);  // Right first
    !
    dfs(node.left, depth + 1, res);
}
```

9. Invert Binary Tree (226)

Mirror the tree (swap all left and right children)

Time: O(n) **Space:** O(h)

Visit each node once to swap. Recursion stack = height h.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;

    // Swap children
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;

    // Recursively invert subtrees
    invertTree(root.left);
    invertTree(root.right);

    return root;
}
```

Super simple! Just 7 lines

! Swap BEFORE recursion, or swap the recursive calls!

Alternative - Iterative BFS:

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        TreeNode node = q.poll();
        TreeNode temp = node.left;
        node.left = node.right;
        node.right = temp;

        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    return root;
}
```

10. Minimum Depth of Binary Tree (111)

Shortest path from root to any leaf

Time: O(n) **Space:** O(w)

BFS stops at first leaf! Queue width = w. Better than DFS O(n) always.

```
public int minDepth(TreeNode root) {
    if (root == null) return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();

        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();

            // Found a leaf - return immediately!
            if (node.left == null && node.right ==
    null) {
                return depth;
            }

            if (node.left != null) q.offer(node.left
    );
            if (node.right != null) q.offer(node.
    right);
        }
    }
    return depth;
```

```
}
```

BFS advantage: Early termination at first leaf!

! DFS must check ALL paths - not optimal here!

! Leaf = both children null, not just one!

11. N-ary Tree Preorder (589)

Root before all children

Time: O(n) **Space:** O(h)

Visit each node once. Stack/recursion depth = height h.

Recursive - Simple:

```
public List<Integer> preorder(Node root) {
    List<Integer> result = new ArrayList<>();
    preord(root, result);
    return result;
}

void preord(Node node, List<Integer> result) {
    if (node == null) return;
    result.add(node.val);  // Root first
    for (Node child : node.children) {
        preord(child, result);  // Then children
    }
}
```

Iterative - Stack:

```
public List<Integer> preorder(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.val);

        // Push children RIGHT to LEFT (stack is LIFO!)
        for (int i = node.children.size() - 1; i >= 0; i--) {
            stack.push(node.children.get(i));
        }
    }
    return result;
}
```

! Stack is LIFO! Must push children in REVERSE order!

! Forgetting reverse gives wrong traversal order!

12. N-ary Tree Postorder (590)

All children before root

Time: O(n) **Space:** O(h)

Recursive:

```
public List<Integer> postorder(Node root) {
    List<Integer> result = new ArrayList<>();
    postord(root, result);
    return result;
}

void postord(Node node, List<Integer> result) {
    if (node == null) return;
    for (Node child : node.children) {
        postord(child, result);  // Children first
    }
    result.add(node.val);  // Root last
}
```

Iterative - Clever Trick:

```
public List<Integer> postorder(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.val);  // Add to front conceptually

        // Push LEFT to RIGHT (no reverse!)
        for (int i = 0; i < node.children.size(); i++) {
            stack.push(node.children.get(i));
        }
    }

    Collections.reverse(result);  // Reverse entire result
    return result;
}
```

Clever trick: Modified preorder + reverse = postorder!

! DON'T reverse children order here! Reverse final result!

13. N-ary Tree Maximum Depth (559)

Longest path from root to leaf

Time: O(n) **Space:** O(h)

Recursive - Clean:

```
public int maxDepth(Node root) {
    if (root == null) return 0;

    int max = 0;
    for (Node child : root.children) {
        max = Math.max(max, maxDepth(child));
    }

    return max + 1;  // Add 1 for current level
}
```

Super clean! Just max of children + 1

! Initialize max to 0, not Integer.MIN_VALUE!

! Leaf nodes (no children) correctly return 1!

Iterative - BFS:

```
public int maxDepth(Node root) {
    if (root == null) return 0;
    Queue<Node> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node node = q.poll();
            for (Node child : node.children) {
                q.offer(child);
            }
        }
    }
    return depth;
}
```

14. N-ary Level Order (429)

N-ary tree nodes grouped by level

Time: O(n) **Space:** O(w)

```
public List<List<Integer>> levelOrder(Node root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<Node> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();  // Capture size!
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            Node node = q.poll();
            level.add(node.val);

            // Add all children
            for (Node child : node.children) {
                q.offer(child);
            }
        }

        result.add(level);
    }
    return result;
}
```

Same pattern as binary tree! Just loop through children

! Enhanced for-loop safe for adding all children at once!

15. Binary Tree Max Depth (104)

Longest path from root to leaf in binary tree

Time: O(n) **Space:** O(h)

Recursive - Simplest:

```java
public int maxDepth(TreeNode root) {
    if (root == null) return 0;

    int left = maxDepth(root.left);
    int right = maxDepth(root.right);

    return 1 + Math.max(left, right);
}
```

Simplest recursive! Just 3 lines

One-liner:

```java
public int maxDepth(TreeNode root) {
    return root == null ? 0 :
           1 + Math.max(maxDepth(root.left),
    maxDepth(root.right));
}
```

Iterative - BFS:

```java
public int maxDepth(TreeNode root) {
    if (root == null) return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node.left != null) q.offer(node.left
    );
            if (node.right != null) q.offer(node.
    right);
        }
    }
    return depth;
}
```

! Both approaches O(n) here - recursive is cleaner!

16. Reverse Linked List (206)

Reverse singly linked list

Time: O(n) **Space:** O(1) iterative / O(n) recursive

Iterative - O(1) space:

```java
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next;  // Save next
        curr.next = prev;           // Reverse link
        prev = curr;                // Move prev
        curr = next;                // Move curr
    }

    return prev;  // New head
}
```

! Must save next BEFORE changing curr.next!

! Return prev, not curr (curr is null at end)!

Recursive - O(n) space:

```java
public ListNode reverseList(ListNode head) {
    if (head == null || head.next == null) return
    head;

    ListNode newHead = reverseList(head.next);

    // Reverse the link
    head.next.next = head;
    head.next = null;  // Prevent cycle!

    return newHead;
}
```

! newHead is SAME across all frames (final head)!

! Must set head.next = null to prevent cycle!

! head.next.next = head reverses ONE link!

17. Number of Islands (200)

Count connected components of 1's in grid

Time: O(m × n) **Space:** O(m × n)

Visit each cell once. DFS depth worst case = all cells (snake shape).

```java
public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;

    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);  // Mark entire
    island
                count++;          // Found one
    island
            }
        }
    }
    return count;
}

void dfs(char[][] grid, int i, int j) {
    // Check bounds and if water/visited
    if (i < 0 || i >= grid.length ||
        j < 0 || j >= grid[0].length ||
        grid[i][j] == '0') {
        return;
    }

    grid[i][j] = '0';  // Mark as visited

    // Explore 4 directions
    dfs(grid, i + 1, j);  // Down
    dfs(grid, i - 1, j);  // Up
    dfs(grid, i, j + 1);  // Right
    dfs(grid, i, j - 1);  // Left
}
```

! Modifies input! Mark visited BEFORE recursive calls!

! Each DFS marks entire island as visited!

! Use visited[][] if can't modify input!

18. Max Area of Island (695)

Find largest connected component area

Time: O(m × n) **Space:** O(m × n)

```java
public int maxAreaOfIsland(int[][] grid) {
    int maxArea = 0;

    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == 1) {
                int area = dfs(grid, i, j);
                maxArea = Math.max(maxArea, area);
            }
        }
    }

    return maxArea;
}

int dfs(int[][] grid, int i, int j) {
    // Base case: out of bounds or water
    if (i < 0 || i >= grid.length ||
        j < 0 || j >= grid[0].length ||
        grid[i][j] == 0) {
        return 0;
    }

    grid[i][j] = 0;  // Mark visited

    // Count: 1 (current) + all 4 directions
    return 1 + dfs(grid, i + 1, j)
             + dfs(grid, i - 1, j)
             + dfs(grid, i, j + 1)
             + dfs(grid, i, j - 1);
}
```

Return value: 1 + sum of all 4 neighbors

! Return 0 for out of bounds, NOT -1!

! Mark visited before recursive calls!

19. Meeting Rooms (252)

Check if person can attend all meetings

Time: O(n log n) **Space:** O(1)

Sort = O(n log n), single pass = O(n). Sort dominates. In-place sort = O(1) extra space.

```
public boolean canAttendMeetings(int[][] intervals)
    {
    if (intervals == null || intervals.length == 0)
    {
        return true;
    }

    // Sort by start time
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    // Check for overlaps
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < intervals[i-1][1]) {
            return false;  // Overlap found
        }
    }

    return true;
}
```

Sort first! Then check consecutive meetings

! Comparator overflow! Use Integer.compare(a[0], b[0]) for safety!

! Overlap if: start[i] < end[i-1], NOT ≤!

20. Valid Palindrome (125)

Check if alphanumeric characters form palindrome

Time: O(n) **Space:** O(1)

Two pointers, single pass through string. No extra space needed.

```
public boolean isPalindrome(String s) {
    if (s == null || s.length() <= 1) return true;

    int left = 0, right = s.length() - 1;

    while (left < right) {
        char l = s.charAt(left);
        char r = s.charAt(right);

        if (Character.isLetterOrDigit(l) &&
            Character.isLetterOrDigit(r)) {
            if (Character.toLowerCase(l) !=
                Character.toLowerCase(r)) {
                return false;
            }
            left++;
            right--;
        } else if (!Character.isLetterOrDigit(l)) {
            left++;   // Skip non-alphanumeric
        } else {
            right--;  // Skip non-alphanumeric
        }
    }

    return true;
}
```

Two pointers! Skip non-alphanumeric characters

! Move only the pointer pointing to non-alphanumeric!

! Case-insensitive comparison!

21. Longest Palindromic Substring (5)

Find longest palindrome substring

Time: O(n^2) **Space:** O(1)

n possible centers, each expands ≤n times. No extra space for expansion.

```
public String longestPalindrome(String s) {
    if (s == null || s.length() < 1) return "";

    int resLen = 0, resIdx = 0;

    for (int i = 0; i < s.length(); i++) {
        // Check odd-length palindromes (center =
    single char)
        int l = i, r = i;
        while (l >= 0 && r < s.length() &&
               s.charAt(l) == s.charAt(r)) {
            if (r - l + 1 > resLen) {
                resIdx = l;
                resLen = r - l + 1;
            }
            l--;
            r++;
        }

        // Check even-length palindromes (center =
    between chars)
        l = i;
        r = i + 1;
        while (l >= 0 && r < s.length() &&
               s.charAt(l) == s.charAt(r)) {
            if (r - l + 1 > resLen) {
                resIdx = l;
                resLen = r - l + 1;
            }
            l--;
            r++;
        }
    }

    return s.substring(resIdx, resIdx + resLen);
}
```

Expand around center! Check both odd and even lengths

! Must check BOTH odd-length (center = i) and even-length (center between i and i+1)!

! Track start index and length, not start and end!

22. Min Cost Climbing Stairs (746)

Minimum cost to reach top of stairs

Time: O(n) **Space:** O(n) / O(1)

Process each step once. Space: O(n) for array, O(1) for optimized.

DP Array - O(n) space:

```
public int minCostClimbingStairs(int[] cost) {
    int n = cost.length;
    int[] dp = new int[n + 1];

    // Base: can start from index 0 or 1
    dp[0] = 0;
    dp[1] = 0;

    for (int i = 2; i <= n; i++) {
        dp[i] = Math.min(cost[i-1] + dp[i-1],  // 1
    step
                         cost[i-2] + dp[i-2]);  // 2
     steps
    }

    return dp[n];
}
```

Space-Optimized - O(1):

```
public int minCostClimbingStairs(int[] cost) {
    int prev2 = 0;  // dp[i-2]
    int prev1 = 0;  // dp[i-1]

    for (int i = 2; i <= cost.length; i++) {
        int cur = Math.min(prev1 + cost[i-1],
                           prev2 + cost[i-2]);
        prev2 = prev1;
        prev1 = cur;
    }

    return prev1;
}
```

Only need last 2 values! Can optimize to O(1) space

! dp[i] uses cost[i-1] not cost[i] - index offset!

! Can start from index 0 OR 1 - both free!

23. Climbing Stairs (70)

Count distinct ways to reach top

Time: O(n) **Space:** O(1)

Linear iteration. Constant space with optimization.

Space-Optimized - O(1):

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1;  // ways(0) = 1
    int prev1 = 1;  // ways(1) = 1

```

```
    for (int i = 2; i <= n; i++) {
        int cur = prev1 + prev2;  // Fibonacci!
        prev2 = prev1;
        prev1 = cur;
    }

    return prev1;
}
```

! Base case returns 1 (one way), NOT 0!

It's Fibonacci! f(n) = f(n-1) + f(n-2)

! ways(0) = 1 and ways(1) = 1 as base cases!

DP Array approach:

```
public int climbStairs(int n) {
    if (n <= 1) return 1;
    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

24. House Robber (198)

Max money without robbing adjacent houses

Time: O(n) **Space:** O(1)

Single pass through array. Constant space with optimization.

Space-Optimized - O(1):

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = nums[0];                    // Max up to i-2
    int prev1 = Math.max(nums[0], nums[1]);    // Max up to i-1

    for (int i = 2; i < nums.length; i++) {
        // Either rob current + skip prev, or skip current
        int cur = Math.max(nums[i] + prev2, prev1);
        prev2 = prev1;
        prev1 = cur;
    }

    return prev1;
}
```

! NOT even/odd pattern! It's optimal choice!

! Choice: rob[i] + rob[i-2] OR rob[i-1]!

! Edge case: single house needs special handling!

DP Array:

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(nums[i] + dp[i-2], dp[i-1])
    ;
    }
    return dp[nums.length - 1];
}
```

25. Coin Change (322)

Minimum coins needed for amount (unbounded)

Time: O(amount × coins) **Space:** O(amount)

Nested loops: amount iterations × coins. DP array size = amount.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];

    // Initialize with "infinity" (amount + 1 is impossible)
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;  // Base case: 0 coins for amount 0

    // Build up from 1 to amount
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], 1 + dp[i - coin]);
            }
        }
    }

    // If still "infinity", impossible
    return dp[amount] > amount ? -1 : dp[amount];
}
```

! Use amount+1 as infinity, NOT Integer.MAX_VALUE (overflow with +1)!

Unbounded knapsack: Forward iteration, can reuse coins

! The "1+" represents using one coin!

! Check if result is still "infinity" - return -1 if impossible!

Top-Down Memoization:

```
public int coinChange(int[] coins, int amount) {
    int[] memo = new int[amount + 1];
    Arrays.fill(memo, -1);
    int result = helper(coins, amount, memo);
    return result == Integer.MAX_VALUE ? -1 : result
    ;
}

int helper(int[] coins, int amount, int[] memo) {
    if (amount == 0) return 0;
    if (amount < 0) return Integer.MAX_VALUE;
    if (memo[amount] != -1) return memo[amount];

    int minCoins = Integer.MAX_VALUE;
    for (int coin : coins) {
        int res = helper(coins, amount - coin, memo)
    ;
        if (res != Integer.MAX_VALUE) {
            minCoins = Math.min(minCoins, 1 + res);
        }
    }

    memo[amount] = minCoins;
    return minCoins;
}
```

26. 0/1 Knapsack

Maximum value with weight limit (each item once)

Time: O(n × W) **Space:** O(n × W) / O(W)

2D: n items × W weights. 1D: Optimized to just W weights.

2D DP - O(n × W) space:

```
public int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i-1] <= w) {
                // Max of: take item OR skip item
                dp[i][w] = Math.max(
                    val[i-1] + dp[i-1][w - wt[i-1]],  // Take
                    dp[i-1][w]                        // Skip
                );
            } else {
                dp[i][w] = dp[i-1][w];  // Can't take
            }
        }
    }

    return dp[n][W];
}
```

1D Optimized - O(W) space:

```
public int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[] dp = new int[W + 1];

    for (int i = 0; i < n; i++) {
        // MUST iterate BACKWARD to prevent reusing item!
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(dp[w],
```

```
                                val[i] + dp[w - wt[i]])
    ;
            }
    }

    return dp[W];
}
```

! BACKWARD iteration prevents reusing same item!

0/1 Knapsack: Each item used at most once - BACKWARD loop!

Unbounded Knapsack: Items can be reused - FORWARD loop!

! Forward loop would allow using same item multiple times!

Algorithm Templates

Backtracking Template

```
void backtrack(state, choices) {
    if (complete) {
        save_solution(state);  // MUST COPY!
        return;
    }

    for (choice : choices) {
        if (valid(choice)) {
            make_choice(state);          // Modify
            backtrack(new_state);         // Recurse
            undo_choice(state);           // RESTORE
    !
        }
    }
}
```

Key: Make → Explore → Undo

BFS Template

```
Queue<Node> q = new LinkedList<>();
q.offer(start);
Set<Node> visited = new HashSet<>();
visited.add(start);

while (!q.isEmpty()) {
    int size = q.size();  // MUST CAPTURE!

    for (int i = 0; i < size; i++) {
        Node node = q.poll();

        if (isGoal(node)) return result;

        for (Node neighbor : getNeighbors(node)) {
            if (!visited.contains(neighbor)) {
                q.offer(neighbor);
                visited.add(neighbor);
            }
        }
    }
}
```

DFS Grid Template

```
void dfs(int[][] grid, int i, int j) {
    // Bounds check + visited check
    if (i < 0 || i >= m || j < 0 || j >= n ||
        grid[i][j] == 0) return;

    grid[i][j] = 0;  // Mark visited FIRST!

    // Explore 4 directions
    dfs(grid, i + 1, j);  // Down
    dfs(grid, i - 1, j);  // Up
    dfs(grid, i, j + 1);  // Right
    dfs(grid, i, j - 1);  // Left
}

// If need to unmark (backtrack):
// Save original value, restore after
```

DP Memoization

```
public int solve(params) {
    // Create memo ONCE
    int[] memo = new int[n];
    Arrays.fill(memo, -1);  // -1 = not computed
    return helper(params, memo);
}

int helper(params, int[] memo) {
    if (base_case) return base_value;

    if (memo[i] != -1) return memo[i];  // Cached!

    // Compute using recursion
    memo[i] = compute_from(helper(smaller, memo));

    return memo[i];
}
```

DP Bottom-Up

```
public int solve(params) {
    int[] dp = new int[n + 1];

    // Initialize base cases
    dp[0] = base_value_0;
    dp[1] = base_value_1;

    // Build up from small to large
    for (int i = 2; i <= n; i++) {
        dp[i] = compute_from(dp[i-1], dp[i-2],
    ...);
    }

    return dp[n];
}
```

Time Complexity Reference

Problem	Time	Space
Letter Combinations	$O(4^n)$	O(n)
Permutations	O(n!)	O(n)
Subsets	$O(n \times 2^n)$	O(n)
Level Order	O(n)	O(w)
Diameter	O(n)	O(h)
Validate BST	O(n)	O(h)
LCA	O(n)	O(h)
Right View	O(n)	O(w)
Invert Tree	O(n)	O(h)
Min Depth	O(n)	O(w)
N-ary Preorder	O(n)	O(h)
N-ary Postorder	O(n)	O(h)
N-ary Max Depth	O(n)	O(h)
N-ary Level Order	O(n)	O(w)
Binary Max Depth	O(n)	O(h)
Reverse List (iter)	O(n)	O(1)
Reverse List (rec)	O(n)	O(n)
Number of Islands	$O(m \times n)$	$O(m \times n)$
Max Area	$O(m \times n)$	$O(m \times n)$
Meeting Rooms	O(n log n)	O(1)
Valid Palindrome	O(n)	O(1)
Longest Palindrome	$O(n^2)$	O(1)
Min Cost Stairs	O(n)	O(1)
Climb Stairs	O(n)	O(1)
House Robber	O(n)	O(1)
Coin Change	$O(amt \times c)$	O(amt)
Knapsack (2D)	$O(n \times W)$	$O(n \times W)$
Knapsack (1D)	$O(n \times W)$	O(W)

Legend: n=size, h=height, w=width, m=rows, n=cols, c=coins, W=weight

---

Remember: Capture queue size! Copy lists! Backward for 0/1! Mark before recurse! Long bounds! Space optimize when fixed lookback!